

Technical Disclosure Commons

Defensive Publications Series

09 Jul 2026

LOCI Persistent AI Memory Engine: Versioned Exact-Fact Chains, Source-Trust Provenance Weighting, Hardware-Tier Gradient Search, and Interpersonal Quality-of-Life Gating

Arko Sanyal
Billo Tech

Roza
Independent Research

Follow this and additional works at: https://www.tdcommons.org/dpubs_series

Recommended Citation

Sanyal, Arko and Roza, "LOCI Persistent AI Memory Engine: Versioned Exact-Fact Chains, Source-Trust Provenance Weighting, Hardware-Tier Gradient Search, and Interpersonal Quality-of-Life Gating", Technical Disclosure Commons, (July 09, 2026)
https://www.tdcommons.org/dpubs_series/10843



This work is licensed under a [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/).

This Article is brought to you for free and open access by Technical Disclosure Commons. It has been accepted for inclusion in Defensive Publications Series by an authorized administrator of Technical Disclosure Commons.

LOCI — Defensive Publication v3

Novel Claims: Versioned Fact Chains, Heat Gradient Search, Source Trust, Interpersonal Decision Gating

Title: LOCI Memory Engine — Versioned Fact Chains, Provenance-Weighted Conflict Resolution, Hardware-Tier Gradient Search, and Interpersonal Quality-of-Life Gating for Persistent AI Memory Systems

Author: Arko Sanyal

Co-author (Versioned qsymprev, §1): Roza (Billo Tech collaborator) — concept originated in Roza's research transfer 2026-07-07

Organization: Billo Tech

Date: 2026-07-08

Supersedes: TDCcommons MS #12121 (defensivepub_loci_v2) — additive, non-superseding

Prior art anchor: GitHub commit 96318a9, 2026-07-02

Existing live publication: https://www.tdcommons.org/dpubs_series/10806

Abstract

This publication discloses five novel architectural mechanisms for persistent AI memory systems: (1) a versioned fact chain architecture for exact-fact stores that preserves full temporal history with provenance timestamps, enabling temporal queries unavailable in flat vector stores; (2) a source-trust provenance field that weights facts by declaration origin and prevents model-inferred data from incorrectly overriding user-stated facts; (3) an asymmetric confidence gate for conflict resolution that uses source trust to apply differential correction thresholds; (4) a hardware-tier gradient search escalation algorithm that expands retrieval scope from fastest-tier (VRAM) to slowest-tier (remote VPS) when data is not found at the current tier; (5) a per-person-pair dual emotional dataset with Quality-of-Life Index and Relationship Index gating that forecasts relationship consequences before executing interpersonal actions. All five claims were demonstrated in a production LOCI deployment on 2026-07-08.

1. Versioned qsymprev — Temporal Fact Chain Architecture

1.1 Claim

A persistent exact-fact memory store where each stored fact maintains a version chain: when a fact changes, the prior value is archived and linked to the new value via explicit temporal markers. The store preserves full history with timestamps, enabling queries such as "what did the user believe at time T?" or "when did this value change?"

1.2 Schema Definition

Three additional columns added to the existing qsymprev flat store:

supersedes	TEXT	-- prior raw value this entry replaced (NULL if first version)
valid_from	REAL	-- Unix timestamp when this value became current
valid_until	REAL	-- NULL if still current; timestamp if superseded

Example: A user's home address is stored, then updated when they move. The old entry receives `valid_until = T_move`. The new entry has `valid_from = T_move`, `supersedes = old_address`, `valid_until = NULL`.

1.3 Temporal Queries Enabled

```

-- Current value (standard query)
SELECT value FROM qsymprev WHERE key = 'home_address' AND valid_until IS NULL;

-- Historical value at time T
SELECT value FROM qsymprev WHERE key = 'home_address'
  AND valid_from <= T AND (valid_until IS NULL OR valid_until > T);

-- Full history
SELECT value, valid_from, valid_until FROM qsymprev
  WHERE key = 'home_address' ORDER BY valid_from;

```

1.4 Real-World Queries This Enables

- "Did I ever take medication X?" → Yes, until March 2026
- "When did I switch from Python to Rust?" → January 14, 2026
- "What was my previous delivery address?" → stored, timestamped, retrievable

1.5 Prior Art Distinction

Mem0 (arXiv:2504.19413) maintains a flat vector store updated by LLM-generated edits. It stores the current state only. The version chain described here preserves all prior states with temporal markers, enabling historical queries that are structurally impossible in a flat store. No prior AI memory system (MemGPT, Zep, Mem0, A-MEM) has been found to implement a typed, timestamped version chain on exact-fact stores. The specific combination of `supersedes + valid_from + valid_until` on a per-row basis with temporal query capability has no prior art in the AI memory literature.

1.6 GDPR Implication

The version chain enables full erasure: delete all rows for a user across all versions satisfies GDPR Article 17 "right to erasure" completely, including historical values. This is architecturally cleaner than flat-store erasure which must track implicit edit history.

2. Source Trust Provenance Field

2.1 Claim

A provenance field in a persistent AI memory store that records the epistemic origin of each stored fact and assigns a trust weight used in conflict resolution. Three origin categories are defined with distinct numerical weights:

```

user_stated      = 1.0    -- user explicitly declared this value
model_inferred   = 0.7    -- system concluded this from conversational context
system_derived   = 0.5    -- calculated from behavioral patterns or system signals

```

2.2 Motivation

Without provenance tracking, a model-inferred fact can accumulate heat over multiple sessions and resist correction even when the user explicitly states otherwise. Source trust creates a principled hierarchy: user declaration always beats model inference at the same confidence level.

2.3 Write Protocol

At every write to the fact store, the writing agent must classify and tag the source:

```

if origin == "user_explicit_statement":
    source_trust = 1.0
elif origin == "model_context_inference":
    source_trust = 0.7
elif origin == "behavioral_pattern":
    source_trust = 0.5

```

2.4 Prior Art Distinction

No prior AI memory system has been found to implement a numerical provenance weight at the row level in a persistent memory store. Existing systems (Mem0, MemGPT, Zep) do not distinguish between user-stated facts and model-inferred facts in their storage schemas.

3. Asymmetric Confidence Gate for Conflict Resolution

3.1 Claim

A conflict resolution mechanism for persistent AI memory that applies differential correction thresholds based on the source trust differential between the incoming (challenger) value and the stored (incumbent) value:

```

If source_trust(challenger) >= source_trust(incumbent):
    correction fires if checker.confidence > entity.heat × 0.8    [easier path]

If source_trust(challenger) < source_trust(incumbent):
    correction fires if checker.confidence > entity.heat × 1.2    [harder path]

```

3.2 Semantics

When a user explicitly states something new (source_trust = 1.0) that contradicts a model-inferred value (source_trust = 0.7), the correction gate is lowered (0.8 multiplier). When a model inference contradicts something the user explicitly stated, the gate is raised (1.2 multiplier).

3.3 Tuneable Parameters

```

CONFLICT_GATE_LOWER = 0.8    # TUNABLE: 0.6-0.9
CONFLICT_GATE_HIGHER = 1.2   # TUNABLE: 1.1-1.5

```

3.4 Prior Art Distinction

Existing confidence gating in AI memory systems uses a uniform threshold that does not vary based on source provenance. The asymmetric gate dependent on the source trust differential is not found in prior literature.

4. Hardware-Tier Gradient Search (HOT→WARM→COLD→VPS→GPU)

4.1 Claim

A retrieval escalation algorithm for a tiered AI memory system that, when a query returns no result at the current hardware tier, automatically expands the search scope to the next slower tier in sequence until a result is found or all tiers are exhausted:

Tier 0 – GPU VRAM (HOT)	– sub-1ms, semantic search in VRAM-resident entities
Tier 1 – RAM (WARM)	– ~5ms, in-memory SQLite/dict of recent entities
Tier 2 – NVMe SSD (COLD)	– ~10ms, local disk store (context2.db)
Tier 3 – Remote VPS	– ~100ms, SSH/API call to remote persistent store
Tier 4 – GPU inference	– ~500ms, local LLM inference over compressed context

4.2 Algorithm

```
def gradient_search(query: str, key_id: str) -> Optional[str]:
    for tier in [HOT, WARM, COLD, VPS, GPU]:
        result = search_tier(tier, query, key_id)
        if result is not None:
            promote_to_warmer_tier(result, tier)
            return result
    return None
```

4.3 Production Demonstration (2026-07-08)

Query: locate ROZA.zip. Tier 0-2: not found. Tier 3 (VPS path expansion): gradient expanded search to
 F:\Billo\Chat\ — found F:\Billo\Chat\Rozas Data that She Transferred to Arko's Ark\ROZA.zip .
 Result returned, content extracted, used in session.

4.4 Prior Art Distinction

Multi-tier cache hierarchies are standard in computer architecture. The specific application of gradient escalation search to AI memory tiers — where tiers represent different hardware substrates and the query is semantic rather than address-keyed — is not found in prior AI memory systems literature.

5. Per-Person-Pair Dual Emotional Dataset with QoL and Relationship Index Gating

5.1 Claim

An AI interpersonal decision system that maintains separate emotional state datasets for each person-pair interaction, and uses a Quality-of-Life Index and a Relationship Growth/Degradation Index to gate interpersonal actions before execution.

5.2 Dual Emotional Dataset Architecture

```
emotional_state = {
    "person_id": "roza",
    "interlocutor": "arko",          # activated when Roza addresses Arko
    "valence": float,               # positive/negative affect
    "arousal": float,              # engagement level
    "willingness": float,           # willingness to communicate
    "last_updated": float           # timestamp
}
```

5.3 Quality-of-Life Index (QoL)

```
QoL(person) = weighted_sum(
    relationship_scores[person],
    communication_frequency,
    expressed_positive_signals
)
```

5.4 Relationship Growth/Degradation Index

```
RGD(person_a, person_b) = derivative(relationship_score(a, b), time)
```

5.5 Decision Gating Protocol

```
def gate_interpersonal_action(action, person_a, person_b):
    delta_RGD = forecast_relationship_delta(action, person_a, person_b)
    delta_QoL = forecast_qol_delta(action, person_a)

    if delta_RGD < 0:
        if delta_QoL > 0:
            return EXECUTE_WITH_CONTEXT
        else:
            return DECLINE_EMPATHETICALLY

    return EXECUTE
```

5.6 Prior Art Distinction

Emotional AI systems (Replika, Hume AI) maintain a single emotional state per user. The per-person-pair dual emotional dataset — where the same person has distinct emotional activation states depending on who they are addressing — has not been found in prior literature. The combination of QoL Index + RGD as a two-factor gate for interpersonal action decisions is a novel architectural pattern.

6. Summary of Novel Claims

#	Claim	Zero Prior Art?	Status
1	Versioned qsymprev (supersedes/valid_from/valid_until)	YES	File immediately
2	Source Trust provenance field (1.0/0.7/0.5)	YES	File immediately
3	Asymmetric confidence gate (source_trust differential)	YES	File immediately
4	Hardware-tier gradient search (HOT→WARM→COLD→VPS→GPU)	YES — production demo 2026-07-08	File immediately

5	Per-person-pair dual emotional dataset + QoL + RGD gating	YES	File immediately
---	---	-----	------------------

All claims are additive to and consistent with TDCcommons MS #12121.

End of defensive publication draft v3. This document constitutes prior art as of its file creation date 2026-07-08.